

● 4단계: 좌표평면과 그래프 (Coordinate Plane & Graphs)

혼돈의 공간에 부여하는 질서와 위치, 그리고 시간에 따른 변화의 궤적

1. 목상과 사유 (철학적·종교적 관점)

좌표(Coordinate)와 그래프(Graph)는 보이지 않는 수학적 관계와 시간에 흐르는 변화를 눈으로 볼 수 있는 형태(Geometry)로 변환해 주는 위대한 번역기입니다.

- **좌표평면: 혼돈의 바다에 선을 그어 질서를 잡다** 아무런 기준도 없고 형태도 없는 광활한 2차원 평면은 마치 창조 이전의 혼돈(Chaos)과 같습니다. 그러나 여기에 가로축(x축)과 세로축(y축)이라는 기준선이 교차하고 원점(0, 0)이 선언되는 순간, 평면 상의 모든 점은 자신만의 유일한 이름표(x, y)를 얻게 됩니다. 종교적으로 이는 창조주가 빛과 어둠, 땅과 바다의 경계를 나누어 세상에 공간적 질서를 창조하신 섭리와 맞닿아 있습니다. 기준이 명확할 때 비로소 우리는 길을 잃지 않고 존재의 정확한 위치를 파악할 수 있습니다.
- **그래프의 선: 시간과 관계가 그려내는 인생의 궤적** 좌표평면 위의 선은 수많은 점들의 모임이자, 하나의 원인(x)에 대응하는 결과(y)들의 연속적인 흐름입니다. 이는 매일의 선택(x)이 엮어내는 우리 삶의 누적된 궤적(Graph)과 같습니다. 인생의 궤적은 때로 정비레 직선처럼 곧고 가파르게 상승하기도 하고, 반비례 곡선처럼 한없이 영(0)에 수렴하는 듯한 고난의 정체기를 거치기도 합니다. 그러나 좌표계 전체를 주관하는 관점에서 보면, 모든 상승과 하강, 곡선과 직선은 전체 삶을 채우는 아름다운 하나의 수학적 조화이자 작품입니다.

2. 왜 사용하는가? 실제 생활에서의 적용점

- **픽셀 우주를 구성하는 2차원 매트릭스: 컴퓨터 그래픽스** 우리가 스마트폰 화면이나 모니터로 마주하는 모든 디지털 이미지는 사실 거대한 정수 격자 좌표계입니다. 화면 왼쪽 위(혹은 왼쪽 아래)를 원점으로 두고 가로 픽셀 좌표 x와 세로 픽셀 좌표 y를 연산하여 특정 좌표에 RGB 색상값을 채워 넣는 코딩 기법은 데카르트의 좌표평면 원리를 그대로 사용한 것입니다.
- **지구 상의 모든 존재를 찾아내는 주소: GPS와 위도·경도** 스마트폰 지도로 맛집을 찾거나 내비게이션으로 목적지를 안내받을 때 작동하는 GPS(위성항법시스템)의 핵심은 지구 표면을 거대한 구면 좌표계로 나누어 놓은 위도(Latitude)와 경도(Longitude)입니다. 두 축이 만나는 점을 좌표로 환산하여 지구 상 어디에 있는 1미터 오차 이내로 나의 정확한 위치를 찾아냅니다.
- **빅데이터의 흐름을 통찰하는 분석의 눈: 데이터 궤적 추적** 주식 시장의 시계열 차트, 기상청의 태풍 예상 경로, 미사일이나 로켓의 포물선 비행 궤적 등은 모두 시간의 축 x에 따른 상태 변수 y의 변화를 추적하여 미래를 예측하는 도구입니다. 좌표와 그래프를 사용하면 복잡한 텍스트 데이터 이면에 숨겨진 본질적인 추세(Trend)를 단번에 간파할 수 있습니다.

1. 데카르트 페인터 (Cartesian Painter)

아래 실습은 2차원 정수 격자 좌표평면(가로 -10 ~ 10, 세로 -10 ~ 10)을 디지털 픽셀 캔버스로 삼아, 정비례 함수 $y = ax$ 와 반비례 함수 $y = \frac{a}{x}$ 의 궤적을 픽셀 단위로 색칠하여 렌더링하는 시각화 툴입니다. 연속적인 직선/곡선이 컴퓨터 격자 모니터 화면에서 어떻게 이산화(Discretize)되어 점들로 찍히는지 직접 체험해 보세요.

🔧 1단계: 도구 준비 (Imports & Settings)

픽셀 캔버스를 모델링하고 시각화하기 위해 필요한 파이썬 핵심 라이브러리를 가져오고, 차트 내부의 한글 인쇄 설정과 수치가 흐트러지지 않도록 마이너스 기호 설정을 완료합니다.

```
In [7]: import platform # 실행 중인 운영체제를 확인하여 맞춤 폰트를 지정하기 위해 임포트합니다.
import numpy as np # 격자 매트릭스 계산 및 배열 생성을 위한 라이브러리입니다.
import matplotlib.pyplot as plt # 격자 픽셀 평면을 드로잉하기 위한 핵심 그래픽 도구입니다.
import ipywidgets as widgets # 슬라이더와 드롭다운 UI 컨트롤러를 만들기 위한 패키지입니다.
from ipywidgets import interact # 사용자의 입력을 실시간으로 플로팅 함수에 반영해주는 가

# 한글이 포함된 그래프 타이틀이나 범례 상자가 네모로 깨져서 나오는 현상을 방지하는 운영체제별 설정입니다.
if platform.system() == "Darwin": # macOS 시스템의 경우
    plt.rcParams["font.family"] = "AppleGothic" # Apple의 기본 한글 고딕 폰트를
elif platform.system() == "Windows": # Windows 시스템의 경우
    plt.rcParams["font.family"] = "Malgun Gothic" # MS의 기본 한글 맑은 고딕 폰트

# 그래프의 축 좌표 범위 등에 마이너스(-) 기호가 사각형 등으로 깨지는 에러를 방지합니다.
plt.rcParams["axes.unicode_minus"] = False
```

🎨 2단계: 격자 평면 연산 및 픽셀 컬러링 함수 정의 (Data Generation & Functions)

정비례/반비례 수식을 계산하고, 정수 격자 좌표 x, y 가 수식의 궤적에 정해진 오차 범위 내로 인접할 때 픽셀에 컬러를 채워 넣어 렌더링하는 `draw_cartesian_pixel_art` 함수를 정의합니다. 기존 1간격의 거친 격자 구조 대신, 간격을 0.01로 100배 촘촘하게 쪼개어 선의 매끄러움과 가독성을 고해상도로 극대화하고 고성능 Numpy 벡터 행렬 연산으로 최적화 처리합니다.

```
In [8]: def draw_cartesian_pixel_art(func_type, a):
    # x축과 y축의 영역을 -10부터 10까지 0.01 간격으로 100배 더 촘촘하게 분할합니다.
    # 이에 따라 해상도는 21x21에서 2001x2001의 초고해상도 디지털 픽셀 캔버스로 확장됩니다.
    grid_x = np.arange(-10, 10.01, 0.01)
    grid_y = np.arange(-10, 10.01, 0.01)

    # 2001x2001 크기의 격자 전체를 연한 회색 바탕색(#f7f7f7)으로 가득 채운 RGB 이미지 행렬.
    canvas = np.ones((len(grid_y), len(grid_x), 3)) * 0.97

    # meshgrid를 활용하여 2차원 좌표 격자 매트릭스를 일괄 생성합니다.
    # X, Y는 2001x2001의 크기를 가지며 각각의 좌표값을 행렬 형태로 담고 있어 고성능 벡터 연산
    X, Y = np.meshgrid(grid_x, grid_y)

    # 정비례 수식 y = ax를 선택한 경우
```

```

if func_type == '정비례 (y = ax)':
    # Y값과 이론적인 수식 연산값(a * X) 사이의 절대 오차 행렬을 계산합니다.
    error = np.abs(Y - a * X)
    # 100배 더 조밀해진 해상도에 비례하여 선의 두께가 예쁘게 표현되도록 0.05 오차 한도 이하
    mask = error <= 0.05
    # 마스크 조건에 해당되는 좌표에 Stark Red (#ff5a5f) 컬러를 일괄 할당합니다.
    canvas[mask] = [1.0, 0.353, 0.373]

# 반비례 수식 y = a/x를 선택한 경우
elif func_type == '반비례 (y = a/x)':
    # 분모가 0이 되어 무한대(inf)나 결측치(nan)가 발생하는 수학적 예외 경고를 무시하도록
    with np.errstate(divide='ignore', invalid='ignore'):
        # 이론적인 반비례 수식 값을 계산합니다.
        target_Y = a / X
        # 실제 Y좌표와의 절대 오차 행렬을 구합니다.
        error = np.abs(Y - target_Y)
        # x=0인 영역(분모가 0)을 필터링하고 오차가 0.065 이하인 영역을 마스크로 선택합니다.
        # 반비례 곡선이 y축에 수렴할 때 선의 뭉개짐을 제어하기 위해 최적의 오차 한도(0.065)를
        mask = (error <= 0.065) & (X != 0)
        # 마스크에 Stark Red (#ff5a5f) 컬러를 채웁니다.
        canvas[mask] = [1.0, 0.353, 0.373]

# 시각화 캔버스 준비 (8x8 크기, 배경은 완전 흰색 #ffffff)
fig, ax = plt.subplots(figsize=(8, 8), facecolor='#ffffff')

# 픽셀 배열(2001x2001 RGB 이미지)을 좌표축 범위 [-10.5, 10.5]에 맞춰 그립니다.
# origin='lower' 옵션을 지정하여 배열의 첫 행이 y의 최하단(-10.5)을 가리키도록 설정,
ax.imshow(canvas, extent=[-10.5, 10.5, -10.5, 10.5], interpolation='nearest')

# 100배 촘촘해진 상태이므로 보조 격자(Grid)는 기존 1 간격으로 적절히 표시하여 시각적 혼선을
ax.set_xticks(np.arange(-10, 11), minor=True)
ax.set_yticks(np.arange(-10, 11), minor=True)
ax.grid(which='minor', color='#e0e0e0', linestyle='-', linewidth=0.8)

# 기준 x축(가로)과 y축(세로)을 3px 두께의 굵은 검정 실선으로 교차시켜 중심을 잡습니다.
ax.axhline(0, color='black', linewidth=3)
ax.axvline(0, color='black', linewidth=3)

# 주요 메인 눈금 범위 설정 (2 간격)
ax.set_xticks(np.arange(-10, 11, 2))
ax.set_yticks(np.arange(-10, 11, 2))
ax.tick_params(labelsize=11)

# 전체 차트 외곽선 테두리를 3px 굵은 검정 실선으로 마무리합니다.
for spine in ax.spines.values():
    spine.set_color('black')
    spine.set_linewidth(3)

# 차트 주변 텍스트 렌더링
ax.set_title(f'데카르트 페인터: {func_type} (a = {a:.1f})', fontsize=15, fontweight='bold')
ax.set_xlabel('x 좌표 (독립변수)', fontsize=12, fontweight='bold')
ax.set_ylabel('y 좌표 (종속변수)', fontsize=12, fontweight='bold')

plt.show()

```

3단계: 인터랙티브 페인터 대시보드 구동 (Plotting & Execution)

앞서 작성한 픽셀 드로잉 함수에 드롭다운 컨트롤러와 실수값 조절 슬라이더를 결합하여, 수식의 변화가 좌표평면 위 픽셀 데이터에 어떻게 실시간으로 색칠되어 나오는지 체험해 봅니다.

```
In [ ]: # interact 함수를 사용하여 함수의 종류(정비례/반비례) 드롭다운 메뉴와 기울기(a) 슬라이더를 결합.
# 100배 촘촘해진 해상도에 맞춰 미세한 기하학적 변화를 조율할 수 있도록 슬라이더 step 단위를 0.01로 설정.
interact(draw_cartesian_pixel_art,
          func_type=widgets Dropdown(options=['정비례 (y = ax)', '반비례 (y = a/x)'], value='정비례 (y = ax)'),
          a=widgets FloatSlider(value=2.0, min=-5.0, max=5.0, step=0.05, description='기울기 a'))

interactive(children=(Dropdown(description='식의 형태', options=('정비례 (y = ax)', '반비례 (y = a/x)'), value='정비례 (y = ax)'),
                    FloatSlider(value=2.0, min=-5.0, max=5.0, step=0.05, description='기울기 a')),
            func_type, a)

Out [ ]: <function __main__.draw_cartesian_pixel_art(func_type, a)>
```

2. 실시간 궤적 추적기 (Trajectory Tracker)

시간에 따라 변화하는 두 물리량의 관계를 매개변수 방정식(Parametric Equation)으로 변환하면 좌표평면 상에 이차식 곡선(포물선)의 형태로 그릴 수 있습니다. 초기 속도 v_0 와 투사 발사 각도 θ 에 따라 중력장 속에서 포물선 궤적을 그리며 움직이는 물체의 $x(t)$, $y(t)$ 위치 변화량을 시뮬레이션하고, 그 기하학적 궤적을 실시간으로 추적하는 물리 대시보드를 구동해 봅니다.

1단계: 도구 준비 및 초기화 (Imports & Settings)

실시간 물리 연산과 시뮬레이션을 구현하기 위해 필요한 라이브러리를 임포트하고 한글 폰트 설정을 다시 한번 로드하여 실습 2의 완전한 독립 실행을 보장합니다.

```
In [10]: import time # 실시간 모션 딜레이를 주는 기능에 사용되는 라이브러리입니다.
import platform # 운영체제 종류별 폰트 식별을 보장하는 라이브러리입니다.
import numpy as np # 포물선 궤적 좌표 수학 계산을 위한 필수 패키지입니다.
import matplotlib.pyplot as plt # 궤적 차트를 렌더링하기 위한 플롯팅 라이브러리입니다.
import ipywidgets as widgets # 슬라이더 GUI 컴포넌트를 제공하는 패키지입니다.
from ipywidgets import interact # 슬라이더 변경 사항을 실시간 그래프 갱신으로 넘겨주는 decorator.

# 한글 폰트가 깨지지 않고 정상 표시되도록 보장하는 OS별 폰트 명세입니다.
if platform.system() == "Darwin":
    plt.rcParams["font.family"] = "AppleGothic"
elif platform.system() == "Windows":
    plt.rcParams["font.family"] = "Malgun Gothic"
plt.rcParams["axes.unicode_minus"] = False
```

2단계: 포물선 운동 방정식 물리 데이터 연산 함수 정의 (Data Generation & Functions)

중력 가속도($g = 9.8 \text{ m/s}^2$) 기준 하에 초기 속도와 발사각 정보를 매개변수 방정식으로 변환하여 시간의 경과에 따른 궤적 좌표를 계산하고, 물리학적인 중요 지점(최고 도달 고도, 수평 도달 거리)을 수학적으로 산출하는 함수를 정의합니다.

In [11]: # 중력가속도 상수를 정의합니다. (9.8 m/s^2)

GRAVITY = 9.8

def calculate_and_plot_trajectory(v0, angle_deg):

입력받은 도 단위(Degree) 각도를 삼각함수 라이브러리 연산을 위해 호도법(Radian)으로 환산
theta = np.radians(angle_deg)

삼각함수를 이용하여 초기 속도(v0)를 가로축 속도성분(v0_x)과 세로축 속도성분(v0_y)으로 나눕니다.
이는 하나의 궤적 벡터를 독립된 두 개의 좌표 축 변화 성분으로 분해하는 과정입니다.

v0_x = v0 * np.cos(theta)

v0_y = v0 * np.sin(theta)

물체가 지면에 다시 닿을 때(y = 0)까지 걸리는 총 체공 시간(Total Flight Time)을 구합니다.

t_flight = 2 * v0_y / g

t_flight = (2 * v0_y) / GRAVITY if GRAVITY > 0 else 0

체공 시간 동안의 타임라인을 200개의 균등 구간으로 쪼개어 세밀한 시간 흐름 배열을 만듭니다.

t_steps = np.linspace(0, t_flight, 200) if t_flight > 0 else np.array([0])

매개변수 t에 의거하여 위치 좌표 x와 y를 연속 계산합니다.

x(t) = v0_x * t (가로 방향으로의 등속도 직선 운동)

y(t) = v0_y * t - 0.5 * g * t^2 (세로 방향으로의 중력을 받는 연직 투사 운동)

x_coords = v0_x * t_steps

y_coords = v0_y * t_steps - 0.5 * GRAVITY * (t_steps ** 2)

물체가 도달하는 최고 높이(Peak Height)와 그때의 시점을 미적분 공식에 의거해 연산합니다.

t_peak = v0_y / g, h_max = v0_y^2 / 2g

t_peak = v0_y / GRAVITY

h_max = (v0_y ** 2) / (2 * GRAVITY)

x_peak = v0_x * t_peak

수평 이동 도달 거리를 최종 계산합니다.

지면에 도달하는 최종 수평 거리는 계산된 x 좌표의 마지막 값입니다.

x_max = x_coords[-1] if len(x_coords) > 0 else 0

시각화 캔버스 준비 (8x6 인치, 배경색은 완전 흰색 #ffffff)

fig, ax = plt.subplots(figsize=(10, 6), facecolor='#ffffff')

ax.set_facecolor('#ffffff')

1. 포물선 비행 궤적 그래프 그리기: 브랜드 메인 컬러인 Stark Red(#ff5a5f)를 사용해 3.5px 굵기의 선을 그립니다.

ax.plot(x_coords, y_coords, color='#ff5a5f', linewidth=3.5, label='비행 궤적')

2. 최고 높이 지점 표시: 물리 연산으로 구한 최고 도달점(x_peak, h_max)에 Accent Green(#008080)을 사용합니다.

두꺼운 검정 테두리(2px)를 씌워 안티 브루탈리즘 디자인 룰을 완성합니다.

ax.plot(x_peak, h_max, marker='o', markersize=10, color='#008080', markeredgewidth=2, label=f'최고점 Height: {h_max:.1f}m (x={x_peak:.1f}m)')

3. 낙하지점(최종 수평 도달점) 표시: x_max 위치의 지면(y = 0) 좌표에 Accent Green(#008080)을 사용합니다.

마찬가지로 2px 굵은 검정 테두리를 씌워 브루탈리즘 스타일을 매칭하고 낙하 지점을 시각적으로 강조합니다.

ax.plot(x_max, 0, marker='o', markersize=10, color='#008080', markeredgewidth=2, label=f'낙하지점 Range: {x_max:.1f}m')

4. 바닥 기준선 지표 그리기: 지면(y = 0)을 나타내는 가로축 기준선을 2.5px 굵기의 검정 선으로 그립니다.

ax.axhline(0, color='black', linewidth=2.5)

ax.axvline(0, color='black', linewidth=2.5)

```

# 5. 데이터 보조 격자(Grid) 추가: 부드러운 회색 점선으로 간격을 식별하기 좋게 만듭니다.
ax.grid(True, color='#e0e0e0', linestyle='--', linewidth=1)

# 6. 눈금 및 뷰포트 범위 고정: 초기 속도가 최대 30m/s이므로, 사거리가 100m 가까이 날아갈
# 그래프 캔버스가 흔들리지 않도록 x축 한계를 0 ~ 100m, y축 한계를 -2 ~ 50m로 고정합니다.
ax.set_xlim(0, 100)
ax.set_ylim(-2, 50)

# 7. 레이블 및 타이틀 정보 렌더링
# 타이틀에 발사각, 초기속도와 함께 최종 수평이동거리(x_max) 결과도 실시간으로 반영하여 표기함
ax.set_title(f'실시간 물리 궤적 추적기 (발사각: {angle_deg}°, 초기속도: {v0}m/s, ...')
ax.set_xlabel('수평 이동 거리 x (m)', fontsize=12, fontweight='bold')
ax.set_ylabel('수직 상승 높이 y (m)', fontsize=12, fontweight='bold')

# 8. 요약 정보 박스 범례 설정: 브랜드 Secondary 컬러인 Vibrant Yellow(#ffd166)로
# 2px의 두꺼운 검정 테두리를 씌워 가독성과 브랜드 아이덴티티를 통일시킵니다.
legend = ax.legend(loc='upper right', fontsize=11, frameon=True)
frame = legend.get_frame()
frame.set_facecolor('#ffd166')
frame.set_edgecolor('black')
frame.set_linewidth(2)

# 9. 그래프 외곽선 굵기 보정: 전체 차트 외부 Spines를 3px 굵은 검정 실선으로 마무리합니다
for spine in ax.spines.values():
    spine.set_color('black')
    spine.set_linewidth(3)

# 계산이 완료되고 디자인된 차트를 렌더링하여 화면에 인쇄합니다.
plt.show()

```

☞ 3단계: 포물선 물리 시뮬레이션 제어 대시보드 구동 (Plotting & Execution)

초기 속도(v_0)와 발사 각도(θ)를 슬라이더 인터페이스로 마음껏 제어하면서, 대수 매개변수 방정식이 그려내는 중력장 모션 궤적의 팽창과 수축, 수평 도달 거리의 연동 방식을 직접 모니터링해 봅니다.

```

In [12]: # interact 함수를 이용하여 calculate_and_plot_trajectory 시뮬레이션 함수에 입력 슬라이더
# 초기 속도 v0는 5m/s에서 30m/s까지 조절할 수 있으며, 발사각은 10도에서 80도까지 정밀 조절이
interact(calculate_and_plot_trajectory,
         v0=widgets.FloatSlider(value=20.0, min=5.0, max=30.0, step=1.0, description='초기 속도 (m/s)'),
         angle_deg=widgets.FloatSlider(value=45.0, min=10.0, max=80.0, step=1.0, description='발사각 (도)'),
         interactive(children=(FloatSlider(value=20.0, description='초기 속도 (m/s)', max=30.0, min=5.0, step=1.0), FloatSlider(value=45.0, description='발사각 (도)', max=80.0, min=10.0, step=1.0))))

Out[12]: <function __main__.calculate_and_plot_trajectory(v0, angle_deg)>

```

3. 한 걸음 더 나아가는 질문 (수학 Retreat 묵상)

실습 대시보드를 직접 경험하고 조작해 본 후, 삶의 궤적과 비즈니스 평면의 맥락으로 확장하여 깊은 사유와 대화를 나누어 봅니다.

1. **질문 1:** 데카르트 페인터에서 수식은 연속적인 선이지만 컴퓨터 격자 캔버스에 그려지는 순간 점들의 끊김(계단 현상)이 발생하는 이산화(Discretization)를 겪게 됩니다. 우리의 이상이나 사명(연속된 완전한 원리)이 일상의 투박한 한계와 실행(이산적인 격자점)으로 번역되어 담길 때, 발생하는 마찰이나 아쉬움을 극복해 보신 경험이 있으신가요?
2. **질문 2:** 2차원 평면 (x, y) 에서 축을 하나 더 추가하면 3차원 공간 (x, y, z) 이 열립니다. 평면적이고 세속적인 삶의 축(예: 소유, 평판) 외에, Joshua님의 인생을 입체적이고 고귀하게 만드는 '제3의 영적·사유적 축'은 무엇이라고 생각하시나요?
3. **질문 3:** 실시간 궤적 추적기에서 발사각 θ 와 초기속도 v_0 는 물체의 탄도를 바꾸는 핵심 주도 요인입니다. 최근 나의 비즈니스 여정이나 마음의 상태를 포물선 궤적으로 매핑해 본다면, 현재 궤적을 수정하기 위해 가장 먼저 재설정해야 할 나의 내면 속 '발사 속도(v_0)'와 '발사 각도(θ)'는 무엇입니까?